THEORY ASSIGNMENT 1

Name VIKASH RAI Roll No.: 22515000162

Branch:B.TECH CSE Section:NA-L2

Q1. HTTP Request–Response Cycle

HyperText Transfer Protocol (HTTP) defines how a client — typically a web browser — and a web server exchange information over the internet. The request-response cycle is the sequence of steps through which a browser asks for a resource and the server delivers it back.

Diagram: Browser → sends HTTP Request → Server → processes the request → sends HTTP Response → Browser renders it

Key Stages:

1. URL Entry: The user types a web address or clicks a hyperlink inside the browser.
2. Request Formation: The browser packages an HTTP request describing the resource needed and the method used, for example GET or POST, and sends it to the destination server.
3. Server Processing: The server accepts the request and works on it, which may involve querying a database or running an application to gather the needed data.
4. Response Assembly: A response is built by the server, typically consisting of a status code, header information, and the actual content requested.
5. Response Delivery: The assembled response travels back across the network to the requesting browser.
6. Rendering: The browser parses the HTML, CSS and JavaScript it received and displays the finished page to the user.

In summary, this cycle of request and reply is what allows browsers and servers to communicate, and it underlies nearly every interaction on the web.

Q2. Introduction to Web Technologies

Building and running websites relies on a set of complementary tools, languages and frameworks. Each of the following plays a distinct role in the overall web development process:

1. HTML (HyperText Markup Language): HTML supplies the skeleton of a webpage — headings, paragraphs, hyperlinks, images, lists, tables and forms are all defined through it, giving the page its basic content and organization.

2. CSS (Cascading Style Sheets): CSS governs how that HTML content is visually presented, covering colors, typography, borders, spacing, alignment and responsive layouts, keeping design decisions separate from structure.

3. JavaScript: JavaScript brings interactivity to a page. It reacts to user input, checks form entries, updates content on the fly, drives animations, and can talk to servers without requiring a full page reload.

4. Bootstrap: Bootstrap is a ready-made frontend framework offering pre-built CSS classes, a responsive grid system and common UI components, letting developers assemble mobile-friendly pages while writing far less custom CSS.

5. jQuery: jQuery is a lightweight JavaScript library that streamlines routine client-side tasks such as DOM manipulation, event handling, animation and AJAX calls, and was once a standard tool for simplifying frontend coding.

6. React.js: React.js is a library for building interactive user interfaces by breaking a page into small, reusable components, which makes large and dynamic frontends easier to construct and maintain.

Q3. Client-Side vs. Server-Side Technologies

Modern web applications rely on both client-side and server-side technologies working in tandem to deliver a complete experience.

Client-Side Technologies: These run inside the user's browser and are concerned mainly with presenting the interface and handling user interaction. The browser downloads the necessary files from the server and executes them locally. HTML, CSS and JavaScript fall into this category.

Server-Side Technologies: These execute on the web server and take care of processing incoming requests, applying business rules, interacting with databases, managing authentication, and generating the final response. Common examples are Node.js, PHP, Python/Django and Java/Spring.

Key Differences:

- Where they run: client-side code executes in the browser, while server-side code executes on the server.
- Purpose: client-side technology focuses on presentation and interaction; server-side technology focuses on processing and data handling.
- Common examples: HTML, CSS and JavaScript on the client; Node.js, PHP and Python on the server.
- Data access: server-side code usually talks directly to a database, while client-side code obtains data indirectly through server APIs.

Together, the client renders the interface the user sees, while the server supplies the logic and data behind it.

Q4. Block-Level Elements in HTML

Block-level elements are HTML tags that automatically break onto a new line and stretch to fill the available width of their parent container. They are the primary building blocks used to divide a page into distinct sections.

Characteristics:

7. They start on a fresh line by default.
8. They typically expand to occupy the full width available to them.
9. They are used to split a page into logical, meaningful blocks.
10. They can hold text as well as other permitted elements.
11. They form the backbone of a page's overall structure and layout.

Examples:

- — a generic container for grouping content.
- <p> — represents a paragraph of text.
- <h1> — represents a top-level heading.
- <section> — groups content around a shared theme.
- <header> — holds introductory or navigational material.

Overall, block-level elements make it possible to organize a page's content in a clear and orderly fashion.

Q5. Semantic HTML Elements

Semantic HTML elements are tags whose very names describe the purpose of the content they wrap, rather than relying on generic, meaning-free containers for everything.

Why They Matter in Development:

12. Readability: They make markup easier for other developers to follow.
13. Accessibility: Assistive technologies such as screen readers can interpret page structure more accurately.
14. SEO: Search engines can better understand what different parts of the page represent.
15. Maintainability: Code that is structured meaningfully is simpler to update later.

16. Clarity: Different regions of a page are clearly distinguished from one another.

Examples:

- <header> — the introductory or top area of a page.
- <nav> — a block of navigation links.
- <main> — the primary content of the document.
- <section> — a themed grouping of content.
- <article> — self-contained, independently distributable content.
- <footer> — the closing/footer area of a page.

Using semantic tags therefore raises the structure, accessibility, readability and overall quality of a webpage.

Q6. Creating a Table Using HTML

Tables in HTML are built with the <table> element; <tr> defines each row, <th> defines a heading cell and <td> defines an ordinary data cell. The code below builds the required table:

```
<!DOCTYPE html>
<html>
<head>
<title>Employee Table</title>
</head>
<body>
<table border="1">
<tr>
<th>Sr. No.</th>
<th>Name</th>
<th>Designation</th>
<th>Department</th>
</tr>
<tr><td>1</td><td>Vikash</td><td>Manager</td><td>HR</td></tr>
<tr><td>2</td><td>Tribhuwan</td><td>Project Manager</td><td>Marketing</td></tr>
<tr><td>3</td><td>Savyasachi</td><td>Developer</td><td>Technical</td></tr>
</table>
</body>
</html>
```

Rendered result:

Sr. No.	Name	Designation	Department
1	Vikash	Manager	HR
2	Tribhuwan	Project Manager	Marketing
3	Savyasachi	Developer	Technical

Q7. Creating a Nested List Using HTML

A nested list is formed by placing one list inside a list item of another list. Below, the outer list contains a single item, 'River', and an inner unordered list holds the names of individual rivers:

```
<!DOCTYPE html>
<html>
<body>
<ul>
<li>River
<ul>
<li>Ganga</li>
```

```
<li>Yamuna</li>
<li>Brahmaputra</li>
<li>Narmada</li>
</ul>
</li>
</ul>
</body>
</html>
```

Q8. Types of CSS

There are three principal ways to attach CSS to an HTML document, and the right choice depends on the size and needs of the page.

1. Inline CSS: Styling is applied directly on an element using its style attribute — handy when only one specific element needs custom styling.

Example: Hello

2. Internal CSS: Rules are placed inside a <style> block, usually within the document's <head>, and apply to that single page.

Example: <style> p { color: blue; } </style>

3. External CSS: Styles live in a separate .css file linked in via the <link> tag. This is best for larger sites since one stylesheet can serve many pages at once.

Example: <link rel="stylesheet" href="style.css">

In short, inline CSS suits single elements, internal CSS suits a lone page, and external CSS is ideal for styling shared across an entire site.

Q9. Practical Understanding

HTML, CSS and JavaScript combine to form the foundation of any modern webpage, each contributing a different layer.

HTML — Structure: HTML lays out what content appears and how it is organized — headings, paragraphs, images, links, tables and forms all come from it.

CSS — Presentation: CSS shapes how that content looks, handling colors, fonts, sizing, spacing, borders, alignment, overall layout and responsiveness.

JavaScript — Behavior: JavaScript makes the page behave dynamically — reacting to clicks, validating form input, updating content live, running calculations, and communicating with servers.

Example: On a login screen, HTML builds the username and password fields, CSS styles and arranges the form neatly, and JavaScript checks the entered details before the form is submitted.

In short: HTML = Structure | CSS = Presentation | JavaScript = Behavior.

Q10. Short Answer

a. What role does Bootstrap play in web development?

Bootstrap supplies a library of ready-to-use CSS classes, a responsive grid system, and pre-styled UI components, enabling developers to build sites that adapt well across desktops, tablets and phones. It also cuts down the custom CSS otherwise needed for common interface pieces like buttons, nav bars, forms and cards.

b. What is jQuery, and why did it see such widespread use?

jQuery is a JavaScript library that simplifies frequent client-side operations — DOM manipulation, event handling, animation and AJAX calls become far easier to code. It gained popularity because it let developers accomplish

common tasks with less code while smoothing over inconsistencies between browsers. Even though modern JavaScript now covers much of the same ground natively, jQuery remains a notable part of web development's history.

c. What is React.js, and what challenge does it address in frontend work?

React.js is a JavaScript library for constructing interactive, dynamic user interfaces. It lets developers break a page down into reusable pieces — buttons, navigation bars, forms, cards — and updates the interface efficiently whenever the underlying data changes. This component-driven approach makes large frontend applications considerably easier to build, reuse, test and maintain.